

Agile Project Documentation

AI Code Reviewer

Student Name	DADISETTY AAKANKSHA
Project	AI Code Reviewer
Framework	Agile / Scrum
Date	March 2026
Total Milestones	4 Milestones (3 Sprints + Final Submission)

M1 Parsing & Preprocessing <i>4 files</i>	M2 Error Detection & AI <i>6 files</i>	M3 Reflex Web App <i>8 files</i>	M4 Final Project <i>All 8 files</i>
---	--	--	---

1. Project Overview

1.1 Project Summary

The AI Code Reviewer is a web-based tool that helps analyze Python code and provide useful feedback. The main idea behind this project is to make code review easier, especially for beginners, by automatically detecting errors and suggesting improvements using AI. The system also identifies common bugs, highlights potential security risks, evaluates code quality, and provides AI-based suggestions using the Groq LLaMA language model.

1.2 Project Vision

The vision of this project is to build a simple and helpful tool that acts like a code reviewer. While working on this project, the aim was to create something that can assist developers in understanding their mistakes and improving their coding skills.

1.3 Project Goals

- To analyze Python code using AST
- To detect syntax errors and basic issues
- To integrate AI for suggesting improvements
- To build a simple and interactive UI using Reflex
- To generate a clear analysis report for users

1.4 Technology Stack

Category	Technology	Purpose
Frontend & Backend	Reflex (Python)	Full-stack web framework
AI / LLM	Groq API — LLaMA 3.1 8B	AI-powered code review
LLM Orchestration	LangChain + langchain-groq	LLM API integration layer
Static Analysis	Python ast module	Syntax parsing & issue detection
Config Management	python-dotenv	API key & environment management
Language	Python 3.10+	Primary development language

2. Agile Methodology

2.1 What is Agile?

Agile is an iterative software development methodology that delivers working software in short cycles called Sprints. Instead of planning everything upfront and delivering at the end, Agile teams build, test, and release incrementally — adapting to feedback at every step.

2.2 Agile Framework Used: Scrum

This project follows the Scrum framework. Scrum organizes work into time-boxed Sprints (1–2 weeks), with defined roles, ceremonies, and artifacts.

Scrum Roles

Role	Responsibility	In This Project
Product Owner	Defines requirements and prioritizes backlog	Mentor / Project Guide
Scrum Master	Removes blockers, facilitates ceremonies	Developer (self-managed)
Development Team	Designs, builds, and tests the product	Developer (intern)

Scrum Ceremonies

Ceremony	Purpose	Frequency
Sprint Planning	Select backlog items to complete in the sprint	Start of each sprint
Daily Standup	What did I do? What will I do? Any blockers?	Daily (15 min)
Sprint Review	Demo completed work to stakeholders	End of each sprint
Sprint Retrospective	Reflect: what went well, what to improve	End of each sprint

3. Product Backlog (User Stories)

User stories describe features from the developer's point of view. Format: "As a developer, I want [feature] so that [reason]."

ID	User Story	Priority	Milestone
US-1	As a developer, I want a code editor so I can paste Python code for review.	High	Milestone 1
US-2	As a developer, I want syntax errors detected so I can fix them before running.	High	Milestone 1
US-3	As a developer, I want unused imports flagged so my code stays clean.	High	Milestone 1
US-4	As a developer, I want a quality score and grade so I know how good my code is.	High	Milestone 1
US-5	As a developer, I want dangerous functions flagged so I avoid security risks.	High	Milestone 2
US-6	As a developer, I want AI suggestions so I can improve my code intelligently.	High	Milestone 2
US-7	As a developer, I want a Security Audit tab to see all critical issues together.	Medium	Milestone 2
US-8	As a developer, I want an AI chat assistant to ask questions about my code.	Medium	Milestone 3
US-9	As a developer, I want an analysis history to track my code quality over time.	Medium	Milestone 3

MILESTONE 1

Parsing & Preprocessing

Sprint 1 — Week 1-2

File	Type	Purpose
code_parser.py	NEW	AST parsing, syntax error detection, code metrics
code_visitor.py	NEW	AST node visitor — extracts variables, functions, imports
requirements.txt	NEW	Project dependencies list
.env	NEW	Environment variable file (GROQ_API_KEY placeholder)

Task	Status
code_parser.py — reads code, finds syntax errors, counts metrics	Done
code_visitor.py — AST node visitor, extracts structure	Done
Quality score (0-100) and letter grade A to F logic	Done
Environment setup and requirements configured	Done

MILESTONE 2

Error Detection & AI Integration

Sprint 2 — Week 3-4

Goal: Add static error detection checks and connect to the Groq AI API for intelligent code suggestions. Builds on Milestone 1's parsing foundation.

Files Submitted in Milestone 2

File	Type	Purpose
code_parser.py	Carried over	From Milestone 1
code_visitor.py	Carried over	From Milestone 1
requirements.txt	Updated	New AI/LangChain dependencies added
.env	Carried over	From Milestone 1
error_detector.py	NEW	7 static checks: unused imports, bare except, eval(), etc.
ai_suggester.py	NEW	Groq API integration — AI review and suggestions

What Was Built

Task	Status
error_detector.py — checks for 7 types of code issues	Done
ai_suggestor.py — connects to Groq LLaMA AI via LangChain	Done
Analysis Report tab with score cards and issue list	Done
Security Audit tab for critical issues (eval, exec)	Done

Challenge 1: langchain.schema import no longer exists. Fixed by changing to langchain_core.messages.

Challenge 2: Reflex foreach does not work with List[Dict]. Fixed using typed rx.Base classes.

Total Files: 6 files

MILESTONE 3

Reflex Web App

Sprint 3 — Week 5

Goal: Build the complete Reflex web application with a full UI — AI chat assistant, code history tab, and the main app entry point. This milestone delivers a working, interactive web app.

Files Submitted in Milestone 3

File	Type	Purpose
code_parser.py	Carried over	From Milestone 1
code_visitor.py	Carried over	From Milestone 1
requirements.txt	Carried over	From Milestone 2
.env	Carried over	From Milestone 1
error_detector.py	Carried over	From Milestone 2
ai_suggester.py	Carried over	From Milestone 2
ai_code_reviewer.py	NEW	Reflex app — all pages, UI, state management
rxconfig.py	NEW	Reflex configuration — app name matches folder

What Was Built

Task	Status
ai_code_reviewer.py — full Reflex web app with all tabs and state	Done
rxconfig.py — app_name matches ai_code_reviewer folder	Done
AI Assistant chat tab with user and AI message bubbles	Done
History tab showing past analyses with score and grade	Done
AI toggle — static-only vs full AI analysis mode	Done

Challenge 1: rxconfig.py had app_name='app' but folder was ai_code_reviewer. Fixed by matching both names.

Challenge 2: AI chat caused websocket disconnect. Fixed by adding yield after each state change in the async send_chat function.

Total Files: 8 files

MILESTONE 4

Final Project Submission

Complete Application — All Milestones Combined

Goal: Submit the complete, fully working AI Code Reviewer application with all 8 files integrated together in the correct folder structure. This is the final deliverable for the Virtual Internship.

Final Project Folder Structure

File Path	Milestone Added	Role in Final App
AI_CODE_REVIEWER/rxconfig.py	M3	Reflex app configuration
AI_CODE_REVIEWER/requirements.txt	M1	All project dependencies
AI_CODE_REVIEWER/.env	M1	GROQ_API_KEY environment variable
AI_CODE_REVIEWER/ai_code_reviewer/__init__.py	M4	Package init file (makes folder a module)
AI_CODE_REVIEWER/ai_code_reviewer/ai_code_reviewer.py	M3	Reflex UI, state management, all pages
AI_CODE_REVIEWER/ai_code_reviewer/ai_suggestor.py	M2	Groq API + LangChain AI integration
AI_CODE_REVIEWER/ai_code_reviewer/code_analyzer.py	M4	Orchestrates full pipeline: parse → detect → score → AI
AI_CODE_REVIEWER/ai_code_reviewer/code_parser.py	M1	AST parsing and syntax error detection
AI_CODE_REVIEWER/ai_code_reviewer/code_visitor.py	M1	AST node visitor — variables, functions, imports
AI_CODE_REVIEWER/ai_code_reviewer/error_detector.py	M2	7 static analysis checks (security + quality)

Final Deliverable Status

Milestone	Achievement	Status
Milestone 1	Core parsing engine — code_parser.py + code_visitor.py	Complete
Milestone 2	Error detection + AI integration — error_detector.py + ai_suggester.py	Complete
Milestone 3	Reflex web app — ai_code_reviewer.py + rxconfig.py	Complete
Milestone 4	Final project — all 8 files running together, complete app submitted	Complete

All 9 User Stories Completed | App runs on http://localhost:3000 | Command: reflex run

5. System Architecture

5.1 Module Structure

The project is organized in a modular way where each file handles a specific task such as parsing, error detection, or AI suggestions.

Module / File	Layer	Responsibility
<code>ai_code_reviewer.py</code>	UI + State	Reflex app — all pages, state management, UI components
<code>code_analyzer.py</code>	Pipeline	Orchestrates the full analysis: parse → detect → score → AI
<code>code_parser.py</code>	Analysis	AST parsing, syntax error detection, and code metrics
<code>code_visitor.py</code>	Analysis	AST node visitor — extracts variables, functions, imports
<code>error_detector.py</code>	Analysis	Static checks: unused imports, bare except, dangerous calls
<code>ai_suggestor.py</code>	AI	Groq API integration — AI review and chat responses
<code>rxconfig.py</code>	Config	Reflex app name and configuration
<code>.env</code>	Config	Stores the GROQ_API_KEY securely

5.2 Analysis Pipeline Flow

When the user clicks Analyze Code, the following pipeline executes:

1. User pastes code into the editor and clicks Analyze Code
2. `State.run_analysis()` is triggered in the Reflex state
3. `code_analyzer.analyze_code()` orchestrates the pipeline:
 - `code_parser.parse_code()` — builds AST, detects syntax errors
 - `code_parser.get_code_metrics()` — counts lines, functions, classes
 - `error_detector.detect_errors()` — runs all static checks
 - `_calculate_score()` — computes quality score 0-100 and grade A-F
 - `ai_suggestor.get_ai_suggestions()` — calls Groq LLaMA API
4. Results are stored in Reflex State and displayed in the Analysis Report tab
5. Analysis is saved to history for future comparison

5.3 Static Analysis Checks

Check	Severity	Description
eval() / exec() calls	Error	Flags dangerous function calls that are security risks
Syntax errors	Error	Code that cannot be parsed by the Python interpreter
Unused imports	Warning	Imports declared but never referenced in the code
Bare except clauses	Warning	except: without specifying exception type — hides errors
Mutable default arguments	Warning	Using list/dict as default arguments — causes subtle bugs
Long functions (>50 lines)	Warning	Functions that should be split for better readability
Missing docstrings	Info	Public functions and classes without documentation

6. Retrospective

At the end of the project I reflected on the experience using three questions:

What went well?	What was difficult?	What will I do better?
- Sprints made big tasks feel manageable - Modular files made debugging easier - The final app looks professional	- Reflex setup on Windows took time - Debugging import and type errors - Started the project late	- Start projects from Day 1 - Check library docs before coding - Test each module separately first

7. Project Summary

Milestone	Main Achievement	Status
Milestone 1	Core parsing engine (code_parser.py + code_visitor.py) — AST analysis, quality scoring	Complete
Milestone 2	Error detection + AI integration (error_detector.py + ai_suggestor.py) — 7 static checks + Groq AI	Complete
Milestone 3	Reflex Web App (ai_code_reviewer.py + rxconfig.py) — full UI with AI chat and history tab	Complete
Milestone 4	Final project — all 8 files + __init__.py integrated, complete app running and submitted	Complete

This project was developed step by step during the internship, and each milestone helped in improving both technical understanding and problem-solving skills.

PROJECT STATUS: SUCCESSFULLY COMPLETED
Submitted for Virtual Internship | 4 Milestones | 9 User Stories | All Files Integrated